Cache Design

CS 2410

Wonsun Ahn

University of Pittsburgh

Oracle Cache

- CPU Cycles = CPU Compute Cycles + Memory Stall Cycles

- **Oracle cache**: a cache that never misses
  - In effect, **Memory Stall Cycles == 0**
  - Impossible, since even with infinite capacity, there are still cold misses
  - But useful to set **bounds** on performance

- Real caches may approach performance of oracle caches but can't exceed

- What metric can we use to compare and evaluate real cache designs?
  - AMAT (Average Memory Access Time)

AMAT (Average Memory Access Time)

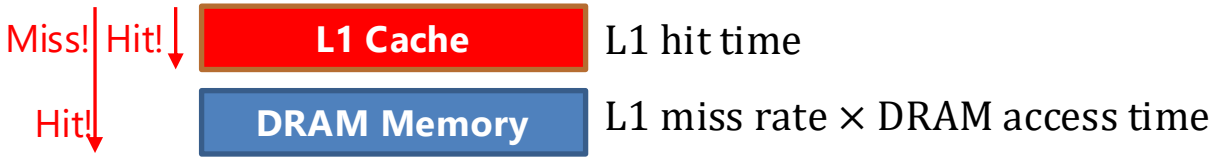
- **AMAT** (Average Memory Access Time) is defined as follows:
  - **AMAT = hit time + (miss rate × miss penalty)**
  - **Hit time:** time to get the data from cache when we hit
  - **Miss rate:** what percentage of cache accesses we miss
  - **Miss penalty:** time to get the data from lower memory when we miss
  - Shouldn't it be **hit rate × hit time** instead of just **hit time**?
    - Hit time is incurred regardless of hit or miss
    - It is more aptly called access time (the time to search for the data)

- Hit time, miss rate, miss penalty are the 3 components of a cache design
  - When evaluating a cache design, we need to consider all 3
  - E.g. A larger cache trade-offs longer hit time for smaller miss rate

AMAT of No-Cache vs. L1 Cache

- For a CPU with no caches:
  - $\text{AMAT(none)} = \text{DRAM access time}$

- For a single-level cache (L1 cache):
  - $\text{AMAT(L1)} = \text{L1 hit time} + (\text{L1 miss rate} \times \text{DRAM access time})$



AMAT of No-Cache vs. L1 Cache

- For L1 Cache to be worth it, AMAT(None) > AMAT(L1) needs to be true.

DRAM Memory	DRAM access time
>?	
L1 Cache	L1 hit time
DRAM Memory	L1 miss rate × DRAM access time

- $\text{AMAT(None)} > \text{AMAT(L1)}?$

$\rightarrow$ DRAM access time > L1 hit time + L1 miss rate × DRAM access time

$\rightarrow$ (1 – L1 miss rate) × DRAM access time > L1 hit time

$\rightarrow$ Benefit from reduced DRAM accesses > Penalty from accessing L1

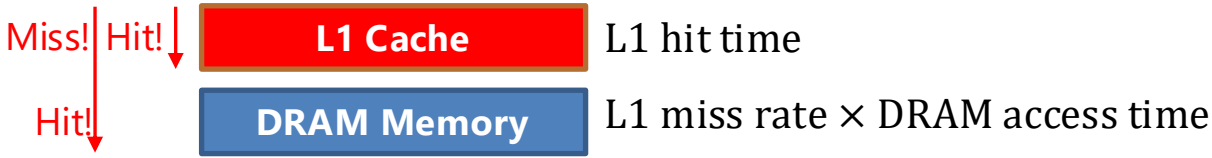
- So, should we add an L1 cache or not?
  - Depends on L1 miss rate, which depends on locality present in software!

AMAT of No-Cache vs. L1 Cache

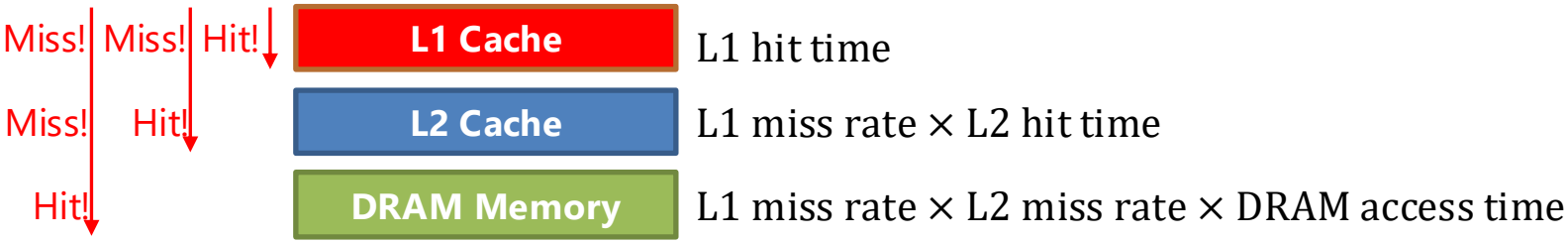
- Let's say CACTI simulation shows a 32 KB L1 cache is feasible with 1 cycle delay
- For L1 cache to be worth it: $(1 - \text{L1 miss rate}) \times \text{DRAM access time} > \text{L1 hit time}$
- If DRAM access time = 100 cycles:
  $(1 - \text{L1 miss rate}) \times 100 > 1 \Rightarrow \text{L1 miss rate} < 0.99$
  - If L1 miss rate can be kept below 99% with 32 KB cache, worth it!
- If DRAM access time = 10 cycles:
  $(1 - \text{L1 miss rate}) \times 10 > 1 \Rightarrow \text{L1 miss rate} < 0.9$
  - If L1 miss rate can be kept below 90% with 32 KB cache, worth it!
- If DRAM access time = 1 cycle:
  $(1 - \text{L1 miss rate}) \times 1 > 1 \Rightarrow \text{L1 miss rate} < 0$
  - The L1 cache will only cause additional delays due to hit time

AMAT of Only L1 Cache vs. L1 + L2 Cache

- For a single-level cache (L1 cache):
  - $\text{AMAT} = \text{L1 hit time} + (\text{L1 miss rate} \times \text{DRAM access time})$



- For a multi-level cache (L1, L2 caches):
  - $\text{AMAT} = \text{L1 hit time} + (\text{L1 miss rate} \times \text{AMAT(L2)})$
  - $\text{AMAT(L2)} = \text{L2 hit time} + (\text{L2 miss rate} \times \text{DRAM access time})$
  - $\text{AMAT} = \text{L1 hit time} + \text{L1 miss rate} \times \text{L2 hit time}$
    $+ \text{L1 miss rate} \times \text{L2 miss rate} \times \text{DRAM access time}$



Cache Design Parameter 1: Cache Capacity

Impact of Cache Capacity on AMAT

- AMAT = hit time + (miss rate × miss penalty)

- Larger caches are **good** for **miss rates**
  - More capacity means you can keep around cache blocks for longer
  - Means you can leverage more of the pre-existing **temporal locality**
  - If entire working set can fit into the cache, no capacity misses!

- But larger caches are **bad** for **hit times**
  - Longer wires and larger decoders mean longer access time

Impact of resizing the L1 cache

- Let's first consider L1 capacity.

L1 Cache	L1 hit time
L2 Cache	L1 miss rate × L2 hit time
DRAM Memory	L1 miss rate × L2 miss rate × DRAM access time

>?

L1′ Cache	L1′ hit time
L2 Cache	L1′ miss rate × L2 hit time
DRAM Memory	L1′ miss rate × L2 miss rate × DRAM access time

- $\mathrm{AMAT}(\mathrm{L1}_{\mathrm{old}}) - \mathrm{AMAT}(\mathrm{L1}_{\mathrm{new}})$
  $= \mathrm{L1\ hit\ time} - \mathrm{L1'\ hit\ time} + (\mathrm{L1\ miss\ rate} - \mathrm{L1'\ miss\ rate}) \times \mathrm{AMAT}(\mathrm{L2})$
  (With the simplifying assumption that L2 miss rate stays the same)

Impact of resizing the last level cache

- Let's first consider L2 capacity.

L1 Cache	L1 hit time
L2 Cache	L1 miss rate × L2 hit time
DRAM Memory	L1 miss rate × L2 miss rate × DRAM access time

>?

L1 Cache	L1 hit time
L2′ Cache	L1 miss rate × L2′ hit time
DRAM Memory	L1 miss rate × L2′ miss rate × DRAM access time

- $\mathrm{AMAT}(\mathrm{L2}_{\mathrm{old}}) - \mathrm{AMAT}(\mathrm{L2}_{\mathrm{new}})$
  $= \mathrm{L2\ hit\ time} - \mathrm{L2'\ hit\ time} + (\mathrm{L2\ miss\ rate} - \mathrm{L2'\ miss\ rate}) \times \mathrm{AMAT(DRAM)}$

Comparing resizing L1 and last level cache

- To recap…
- $\mathrm{AMAT}(\mathrm{L1}_{\mathrm{old}}) - \mathrm{AMAT}(\mathrm{L1}_{\mathrm{new}}) > 0$ equates to:
  $\mathrm{L1\ hit\ time} - \mathrm{L1'\ hit\ time} + (\mathrm{L1\ miss\ rate} - \mathrm{L1'\ miss\ rate}) \times \mathrm{AMAT}(\mathrm{L2}) > 0$
- $\mathrm{AMAT}(\mathrm{L2}_{\mathrm{old}}) - \mathrm{AMAT}(\mathrm{L2}_{\mathrm{new}}) > 0$ equates to:
  $\mathrm{L2\ hit\ time} - \mathrm{L2'\ hit\ time} + (\mathrm{L2\ miss\ rate} - \mathrm{L2'\ miss\ rate}) \times \mathrm{AMAT}(\mathrm{DRAM}) > 0$
- Since AMAT(L2) < AMAT(DRAM)
  - Hit time is more important for L1 → smaller cache
  - Miss rate is more important for L2 → larger cache

What cache size(s) should I choose?

- How should each cache level be sized?

- **Ideally**, cache levels and sizes would **reflect working set sizes**.
  - So that each level of cache leverage a level of temporal locality. E.g.:
    - L1: Small set of data accessed very frequently (typically stack variables)
    - L2: Medium set of data (actively accessed part of data structure)
    - L3: Larger set of data (entire data structure)
    - Memory: All program data

- Simulate multiple cache levels and sizes and choose one with lowest AMAT
  - Simulate on the applications that you care about
  - In the end, it must be a **compromise** (giving best average AMAT)

Cache Design Parameter 2: Cache Block Size

Impact of Cache Block Size on AMAT

- AMAT = hit time + (miss rate × miss penalty)

- **Cache block** (a.k.a. **cache line**)
  - Unit of transfer for cache data (typically 32 or 64 bytes)
  - If program accesses any byte in cache block, entire block is brought in
  - Each level of a multi-level cache can have a different cache block size

- Impact of larger cache block size on **miss rate**
  - Maybe **smaller miss rate** due to **better** leveraging of **spatial locality**
  - Maybe **bigger miss rate** due to **worse** leveraging of **temporal locality** (Bringing in more data at a time may push out other useful data)

- Impact of larger cache block size on **miss penalty**
  - With a limited bus width, may take multiple transfers for a large block
  - E.g. DDR 4 DRAM bus width is 8 bytes, so 8 transfers for 64-byte block
  - Could lead to **increase in miss penalty**

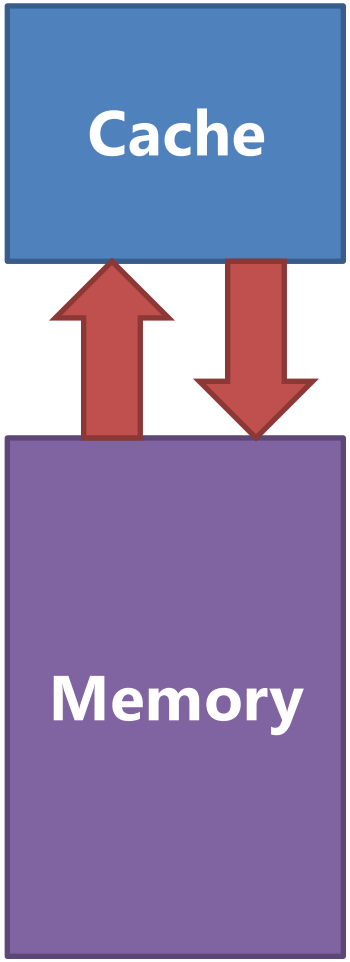
Cache Block Size impact on Miss Penalty

- On a miss, the data must come from lower memory
- Block size determines block transfer time:

$$\frac{\text{seconds}}{\text{block}} = \underbrace{\frac{\text{seconds}}{\text{byte}}}_{\text{byte transfer time}} \times \underbrace{\frac{\text{bytes}}{\text{block}}}_{\text{block size}}$$

$$= \frac{1}{\underbrace{\frac{\text{cycles}}{\text{second}}}_{\text{bus cycle time}} \times \underbrace{\frac{\text{bytes}}{\text{cycle}}}_{\text{bus width}}} \times \underbrace{\frac{\text{bytes}}{\text{block}}}_{\text{block size}}$$

- Has both latency and bandwidth implications



What cache block size should I choose?

- Again, that depends on the application
  - How much spatial and temporal locality the application has

- Simulate multiple cache block sizes and choose one with lowest AMAT
  - Simulate on benchmarks that you care about and choose best average
  - You may end up with different block sizes for each level of the cache

Cache Design Parameter 3: Cache Associativity

Mapping blocks from memory to caches

- Cache size is much smaller compared to the entire memory space
  - Must map all the blocks in memory to limited CPU cache
- Does this sound familiar? Remember branch prediction?
  - Had similar problem of mapping PCs to a limited BHT
  - What did we do then?
    - We hashed PC to an entry in the BHT
    - On a hash conflict, we replaced old entry with more recent one

- We will use a similar idea with caches
  - **Hash memory addresses** to entries in cache
  - On a conflict:
    - **Replace** old cache block with more recent one
    - Or, **chain** multiple cache blocks on to same hash entry

Impact of Cache Associativity on AMAT

- Depending on hash function and chaining, a cache is either:
  - **Direct-mapped** (**no chaining** allowed)
  - **Set-associative** (**some chaining** allowed)
  - **Fully-associative** (**limitless chaining** allowed)

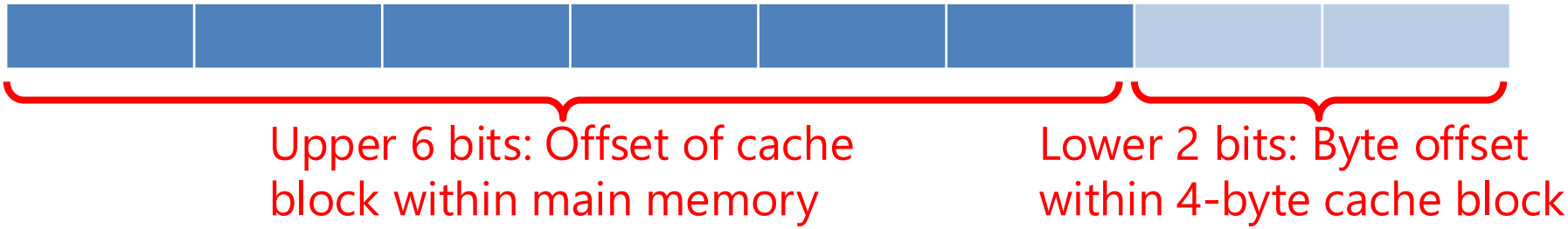
- Impact of more associativity on **miss rate**
  - **Smaller miss rate** due to less misses due to hash conflicts
  - Misses due to hash conflicts are called **conflict misses**
    - A third category of misses besides cold and capacity misses

- Impact of more associativity on **hit time**
  - **Longer hit time** due to need to search through long chain

Direct-mapped Caches

Assumptions

- Let's assume for the sake of concise explanations
  - 8-bit memory addresses
  - 4-byte (one word) cache block sizes
- Of course these are not typical values. Typical values are:
  - 32-bit or 64-bit memory addresses (32-bit or 64-bit CPU)
  - 32-byte or 64-byte cache blocks sizes (for spatial locality)
  - But too many bits in addresses are going to give you a headache
- According to our assumption, here's a breakdown of address bits



  - When I refer to addresses, I will sometimes omit the lower 2 bits (When we talk about cache block transfer, that part is irrelevant)

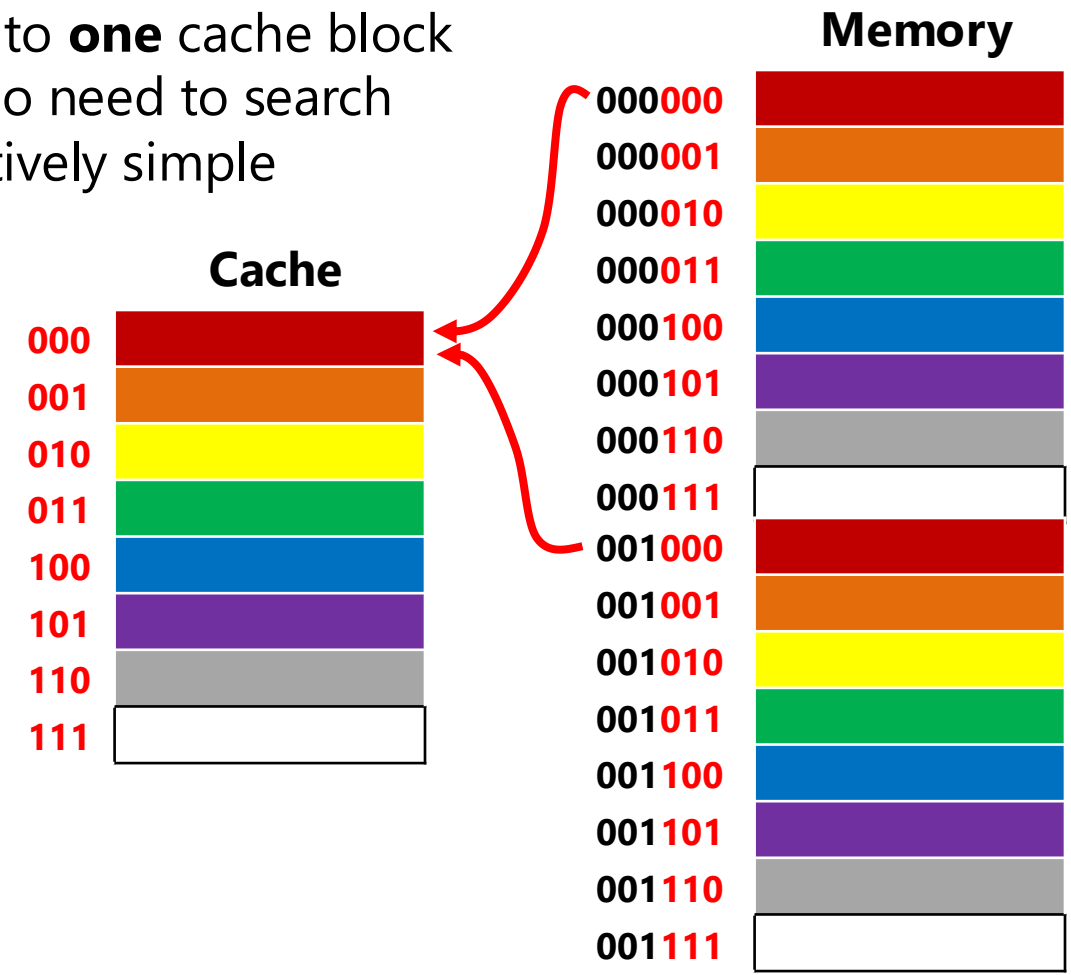
Direct-mapped Cache Hash Function

- Each memory address maps to **one** cache block
  - No chaining allowed so no need to search
  - Implementing this is relatively simple

Hash function:
For this 8-entry cache, to find **cache block index**, take the lowest 3 cache block offset bits in address.

But if our program accesses **001000**, then **000000**, how do we tell them apart?

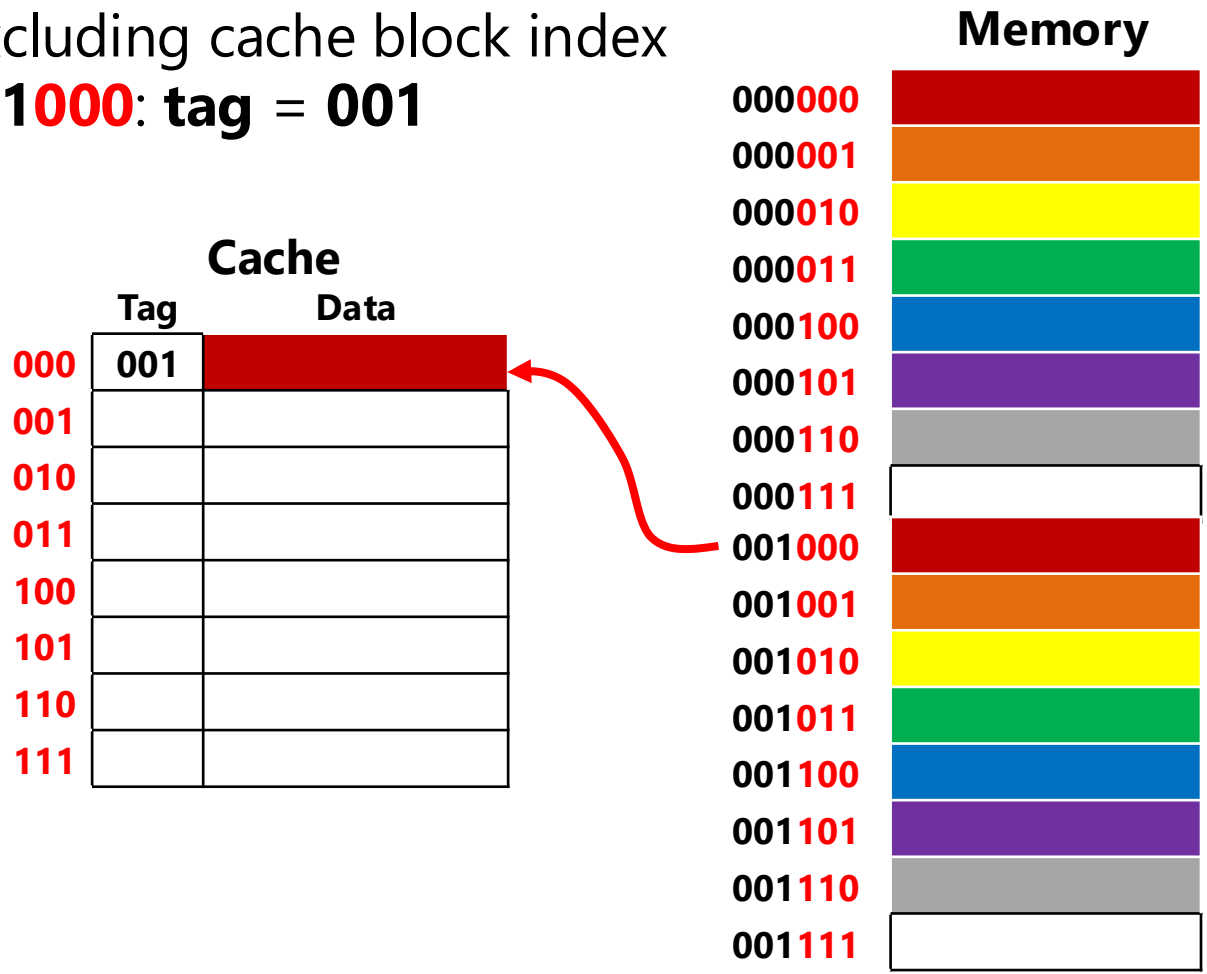
Tags!



Tags help differentiate between conflicting blocks

Tag: part of address excluding cache block index

- On allocation of **001000**: **tag** = **001**



Tags help differentiate between conflicting blocks

Tag: part of address excluding cache block index

- On allocation of **001000**: **tag** = **001**
- On allocation of **000000**: **tag** = **000**

Cache

	Tag	Data
000	000	
001		
010		
011		
100		
101		
110		
111		

Memory

000000	
000001	
000010	
000011	
000100	
000101	
000110	
000111	
001000	
001001	
001010	
001011	
001100	
001101	
001110	
001111	

University of Pittsburgh

Valid bit indicates that block contains valid data

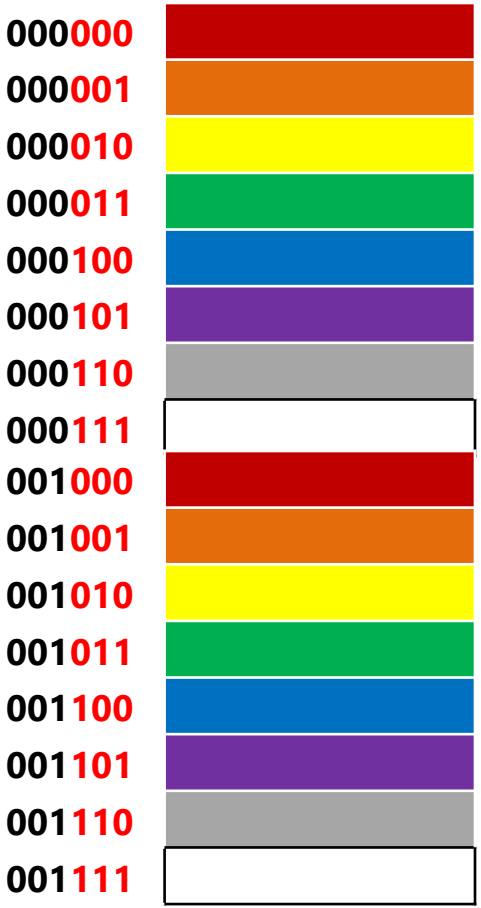
Valid bit: indicates that the block is valid

- Set to 0 initially when cache block is empty
- Set to 1 when a cache block is allocated

Cache

	V	Tag	Data
000	1	000	
001	0		
010	0		
011	0		
100	0		
101	0		
110	0		
111	0		

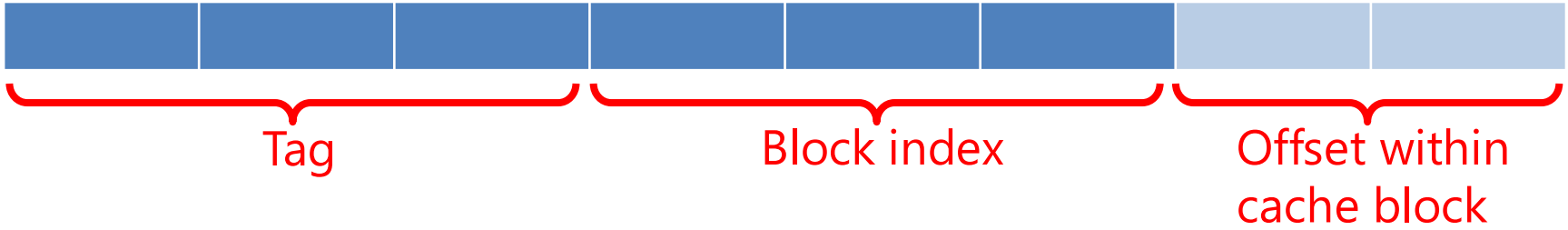
Memory



- Cache hit: V == 1 &&
  CacheBlock.Tag == MemoryBlock.Tag

Quiz: Address Bits Breakdown

- Now with the following parameters:
  - 8-bit memory addresses
  - 4-byte cache block sizes
  - 8-block cache
- How would we breakdown the memory address bits?



  - First, the correct cache block is accessed using the **block index**
  - Then, the **tag** is compared to the cache block tag
  - If matched, **offset** is used to access specific byte within block

Example: A Direct-mapped Cache

- When the program first starts, we **set all the valid bits to 0**.
  - Signals all cache lines are empty
- Now let's try a sequence of reads... do these **hit** or **miss?** How do the cache contents change?

000000	**miss**	Cold misses
100101	**miss**	
100110	**miss**	
100101	**hit**	
010000	**miss**	← Cold miss
000000	**miss**	← Capacity miss?

tag index

	V	Tag	Data
000	**1**	**010**	**something**
001	**0**		
010	**0**		
011	**0**		
100	**0**		
101	**1**	**100**	**something**
110	**1**	**100**	**something**
111	**0**		

Conflict Misses

- What should we call 2nd miss on **000000**?
  - Awkward to call it a capacity miss (It's not like capacity was lacking)
  - Let's call it a **conflict miss**

000000 **miss**
100101 **miss**
100110 **miss**
} Cold misses

100101 **hit**

010000 **miss** ← Cold miss

000000 **miss** ← Capacity miss? Conflict miss!

tag **index**

	V	Tag	Data
000	1	010	something
001	0		
010	0		
011	0		
100	1	100	something
101	1	100	something
110	0		
111	0		

University of Pittsburgh

Types of Cache Misses (Revised)

- Besides cold misses and capacity misses, there are conflict misses

- **Cold miss** (a.k.a. **compulsory miss**)
  - Miss suffered when data is accessed for the **first time** by program

- **Capacity miss**
  - Miss on a **repeat access** suffered due to a lack of **capacity**
  - When the program's **working set is larger than can fit in the cache**

- **Conflict miss**
  - Miss on a **repeat access** suffered due to a lack of **associativity**
  - **Associativity**: degree of freedom in associating cache block with address
  - Direct mapped caches have no associativity
    - Since cache blocks are directly mapped to a particular block index

Associative caches

Flexible block placement

- Direct-mapped caches can have lots of **conflicts**
  - Multiple memory locations "fight" for the same cache line
- Assume:
  1. Cache is 4-block direct-mapped
  2. Following strided memory access pattern:
     - 0, 16, 32, 48
     - 00000000, 00010000, 00100000, 00110000

	V	Tag	Data
00	1	0011	
01	0		
10	0		
11	0		

- **What would happen?**
  - They all land on the same index, and kick each other out.
  - Those other 3 blocks are not even getting used!
  - What if we used the space to chain conflicting blocks?

Full associativity

- Let's make our 4-block cache **4-way set-associative**.

V	Tag	D
1	000000	

V	Tag	D
1	001100	

V	Tag	D
1	000100	

V	Tag	D
1	001000	

- What's the difference?
  - Now a hashed location can be associated with **any** of the 4 blocks
  - Analogous to having a hash conflict chain 4-entries long
  - The 4 cache blocks are said to be part of a cache **set**
  - When set size == cache size, it is said to be **fully associative**
- Let's do that sequence of reads again: 0, 16, 32, 48, 0, 16, 32, 48...
- Notice tag is now bigger, since no index bits are needed for selection
  - Index is called **set index** in this context (since it designates a set)
- Now cache holds the entire **working set**: no more misses!

Example: A 2-way Set-Associative Cache

- 8-block 2-way set-associative cache (same size but more associative)
- Let's try the same stream of accesses as direct-mapped cache
- Yay! 2nd access to **000000** is no longer a conflict miss!

000000 miss
100101 miss
100110 miss
100101 hit
010000 miss
000000 hit

tag index

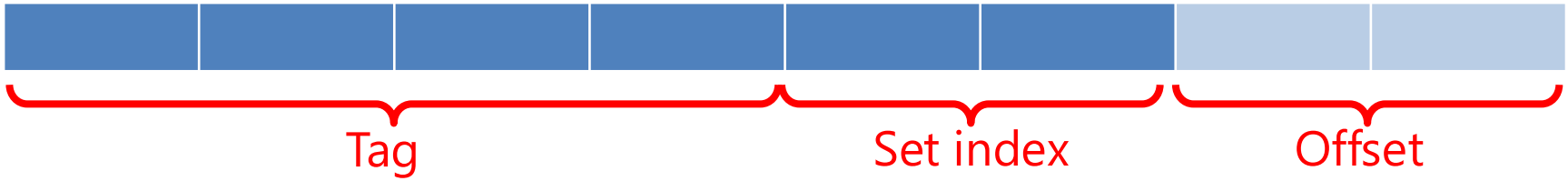
Set	V	Tag	Data	V	Tag	Data
00	1	0000	something	1	0100	something
01	1	1001	something	0		
10	1	1001	something	0		
11	0			0		

University of Pittsburgh

Address Bits Breakdown

- A fully associative cache (doesn't matter how many blocks):

- With 16-block 4-way set-associative cache:



  - 16 / 4 = 4 sets in cache. So, 2 bits required for set index.
- With 64-block 4-way set-associative cache:

  - 64 / 4 = 16 sets in cache. So, 4 bits required for set index.

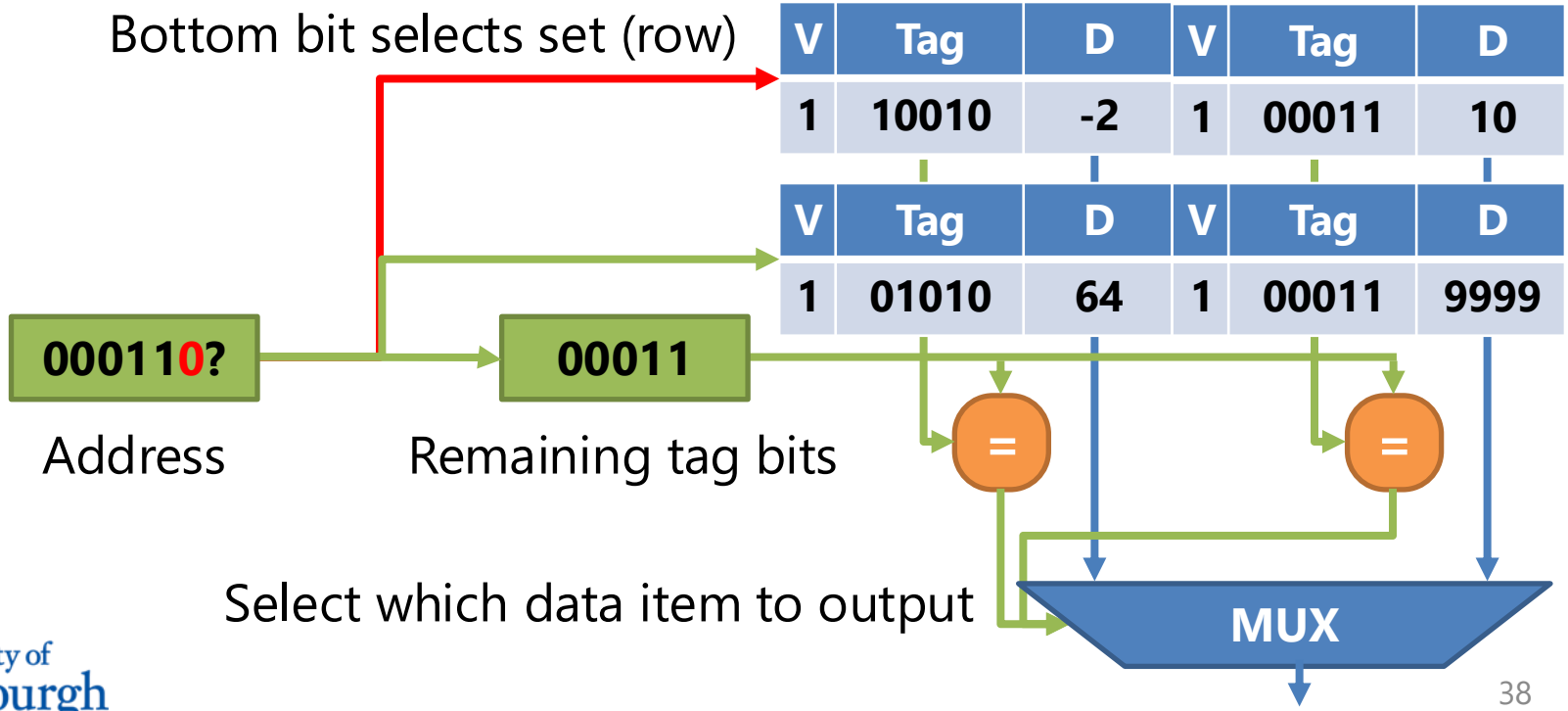
Want More Examples?

- Try out the Cache Visualizer on the course github:
  - https://github.com/wonsunahn/CS2410_Spring2026/tree/main/resources/cache_demo
  - Courtesy of Jarrett Billingsley

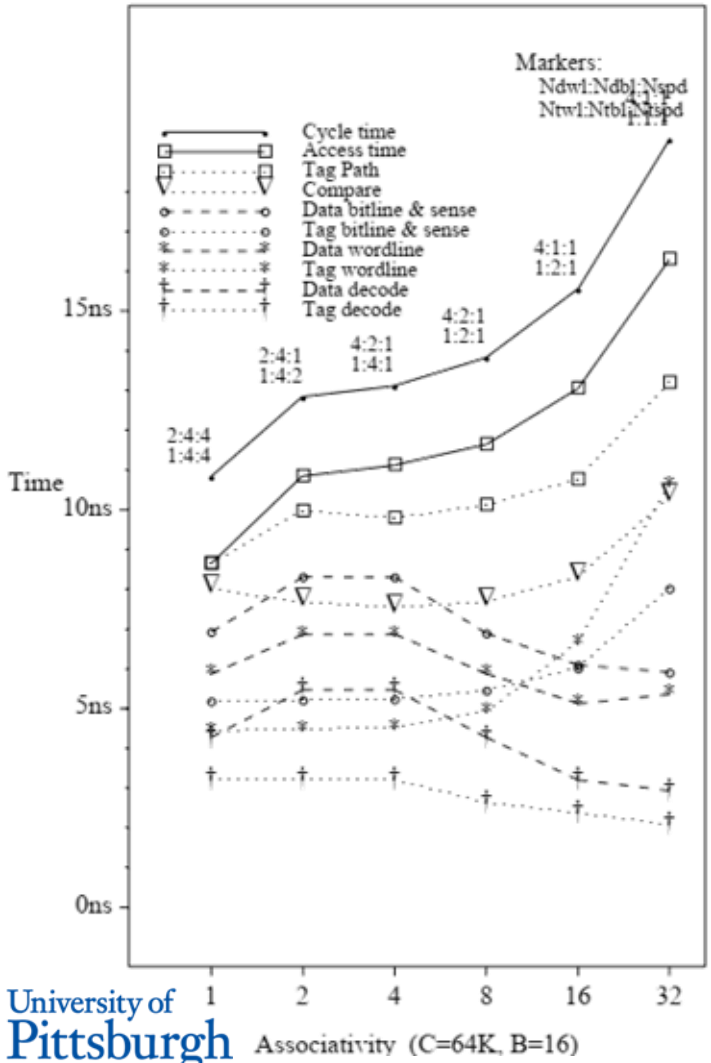
- Visualizes cache organization for various parameters
  - Cache block size
  - Number of blocks in cache (capacity)
  - Cache associativity

Associativity is Costly

- Associativity requires complex circuitry and **increases hit time**
- Full associativity only used when a miss is extremely costly
- Typically, caches are direct mapped or 2-, 4-, 8-way set-associative



Access/cycle time as a function of associativity



Steven J.E. Wilton and Norman P. Jouppi, "An Enhanced Access and Cycle Time Model for On-Chip Caches", WRL Research Report 93/5, 1994.

- First paper to introduce **CACTI**, a simulator for memory structures
  - Still used today to model caches
- Components that increase delay
  - Compare (of tags)
  - Tag wordline & bitline delays

Cache Design Parameter 4: Cache Replacement Policy

Cache Replacement

- If we have a cache miss and no empty blocks, what then?

V	Tag	D
1	000000	

V	Tag	D
1	001100	

V	Tag	D
1	000001	

V	Tag	D
1	001000	

- Let's read memory address 4 (**000001**00).
  - Uh oh. That's a miss. Where do we put it?
- With associative caches, you must have a **replacement scheme.**
  - Which block to evict (kick out) when you're out of empty slots?
- The simplest replacement scheme is **random**.
  - Just pick one. Doesn't matter which.
- What would make more sense?
  - How about taking **temporal locality** into account?

LRU (Least-Recently-Used) Replacement

- When you need to evict a block, kick out the oldest one.

V	Tag	Age
1	000000	3

V	Tag	Age
1	001100	0

V	Tag	Age
1	000100	2

V	Tag	Age
1	001000	1

- Our read history is: 000000, 000100, 001000, 001100.
- Our next read is to 000001. Which block should we replace?

LRU (Least-Recently-Used) Replacement

- When you need to evict a block, kick out the oldest one.

V	Tag	Age
1	000001	0

V	Tag	Age
1	001100	1

V	Tag	Age
1	000100	3

V	Tag	Age
1	001000	2

- Our read history is: 000000, 000100, 001000, 001100.
- Our next read is to 000001. Which block should we replace?
  - **Least recently used** since it is not likely to be used any time soon. (Assuming that the program has **temporal locality**.)
  - On a cache hit, reset counter to 0 (most recently used)
  - For other blocks in set, age by incrementing saturating counter

Impact of LRU on AMAT

- AMAT = hit time + (miss rate × miss penalty)

- Impact of LRU on **miss rate**
  - **Smaller miss rate** due to better leveraging of **temporal locality**
    Larger miss rate due to decrease in capacity due to **added metadata**
    - Saturating counter for LRU uses bits and adds to metadata
    - The **valid bit**, **tag**, and **saturating counter** are all metadata
    - Additional bits for metadata comes in expense of bits for real data
      - Need to limit the number of bits in counter
      - But with reduced bits counter will saturate early, reducing accuracy

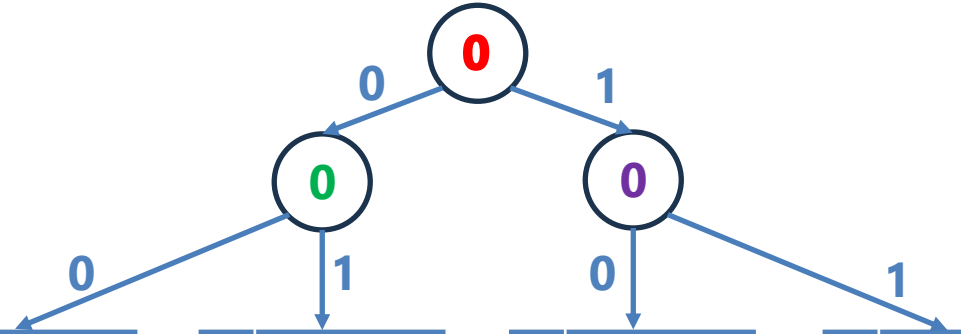
- Modern CPUs use schemes like PLRU (Pseudo-LRU) to save on bits

PLRU (Pseudo-LRU) Replacement

- Our read history is: 000000, 000001, 00010, 000011.

PLRU	V	Tag	V	Tag	V	Tag	V	Tag
000	0		0		0		0	

- Each bit in PLRU field correspond to nodes in binary tree
  - Tells direction to traverse to find the Least Recently Used block



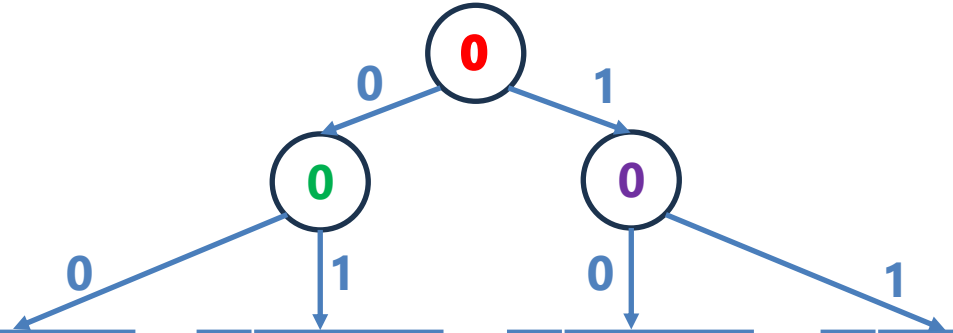
PLRU	V	Tag	V	Tag	V	Tag	V	Tag
000	1	000000	1	000001	1	000010	1	000011

PLRU (Pseudo-LRU) Replacement

- Our read history is: 000000, 000001, 00010, 000011.

PLRU	V	Tag	V	Tag	V	Tag	V	Tag
000	0		0		0		0	

- Our next read is to 000000. Which block should we choose?



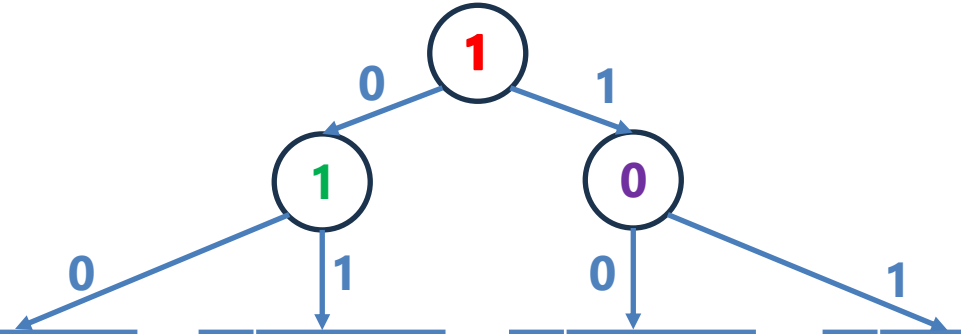
PLRU	V	Tag	V	Tag	V	Tag	V	Tag
000	1	000000	0		0		0	

PLRU (Pseudo-LRU) Replacement

- Our read history is: 000000, 000001, 00010, 000011.

PLRU	V	Tag	V	Tag	V	Tag	V	Tag
000	0		0		0		0	

- Our next read is to 000000. Which block should we choose?
  - Then update PLRU bits so they point opposite direction traversed.



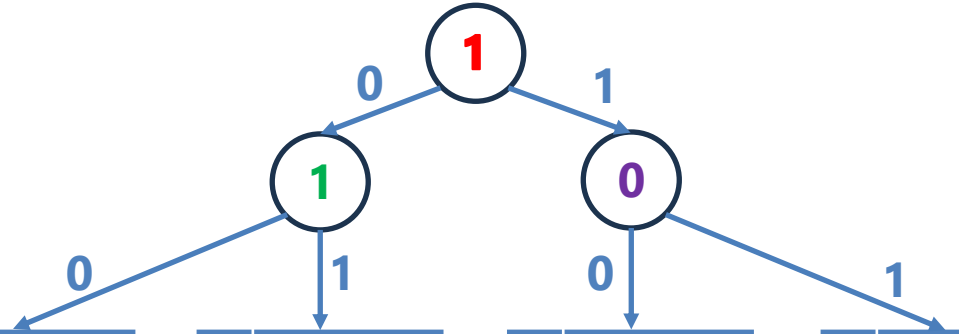
PLRU	V	Tag	V	Tag	V	Tag	V	Tag
110	1	000000	0		0		0	

PLRU (Pseudo-LRU) Replacement

- Our read history is: 000000, 000001, 00010, 000011.

PLRU	V	Tag	V	Tag	V	Tag	V	Tag
000	0		0		0		0	

- Our next read is to 000001. Which block should we choose?



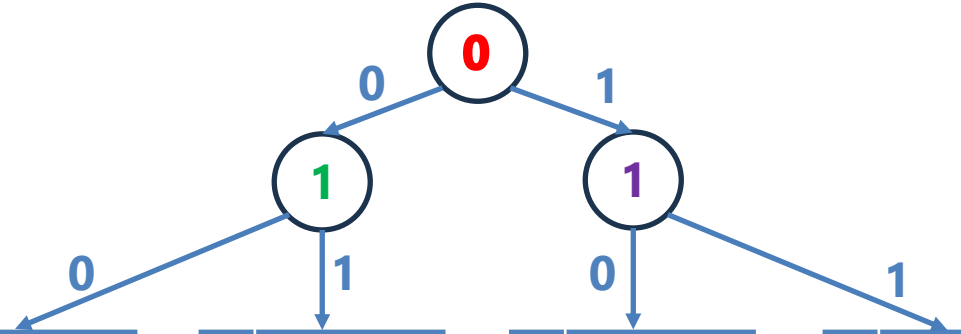
PLRU	V	Tag	V	Tag	V	Tag	V	Tag
110	1	000000	0		1	000001	0	

PLRU (Pseudo-LRU) Replacement

- Our read history is: 000000, 000001, 00010, 000011.

PLRU	V	Tag	V	Tag	V	Tag	V	Tag
000	0		0		0		0	

- Our next read is to 000001. Which block should we choose?
  - Then update PLRU bits so they point opposite direction traversed.



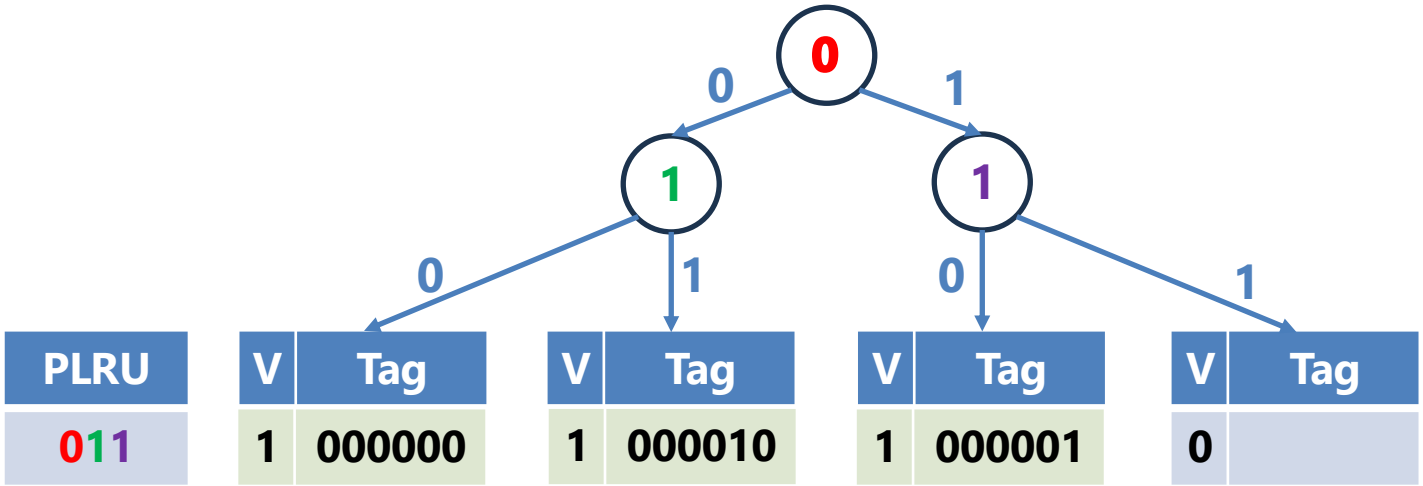
PLRU	V	Tag	V	Tag	V	Tag	V	Tag
011	1	000000	0		1	000001	0	

PLRU (Pseudo-LRU) Replacement

- Our read history is: 000000, 000001, 00010, 000011.

PLRU	V	Tag	V	Tag	V	Tag	V	Tag
000	0		0		0		0	

- Our next read is to 000010. Which block should we choose?

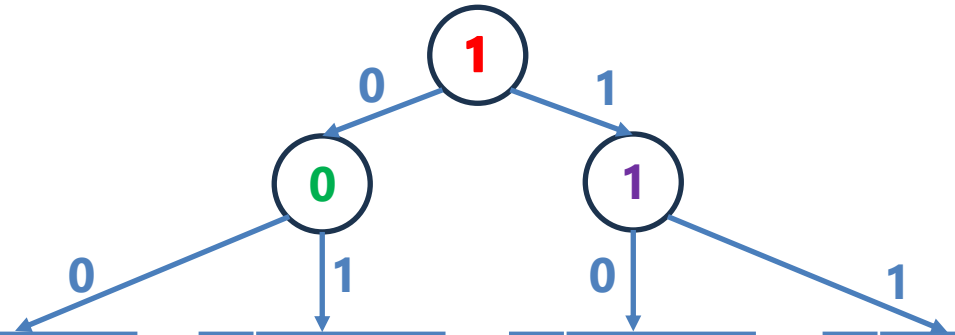


PLRU (Pseudo-LRU) Replacement

- Our read history is: 000000, 000001, 00010, 000011.

PLRU	V	Tag	V	Tag	V	Tag	V	Tag
000	0		0		0		0	

- Our next read is to 000010. Which block should we choose?
  - Then update PLRU bits so they point opposite direction traversed.



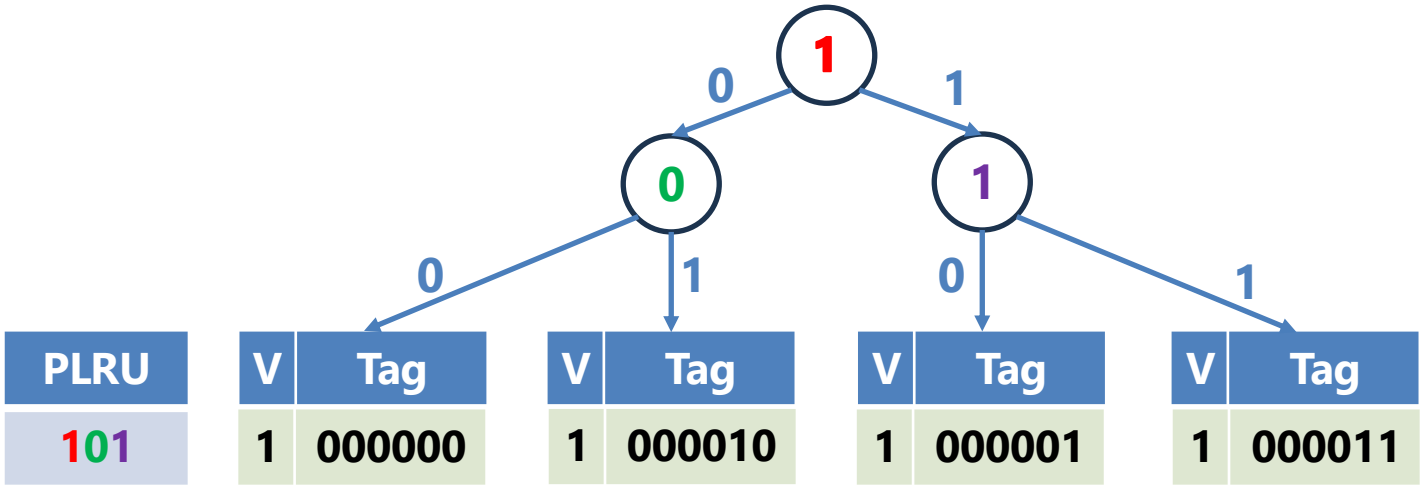
PLRU	V	Tag	V	Tag	V	Tag	V	Tag
101	1	000000	1	000010	1	000001	0	

PLRU (Pseudo-LRU) Replacement

- Our read history is: 000000, 000001, 00010, 000011.

PLRU	V	Tag	V	Tag	V	Tag	V	Tag
000	0		0		0		0	

- Our next read is to 000011. Which block should we choose?

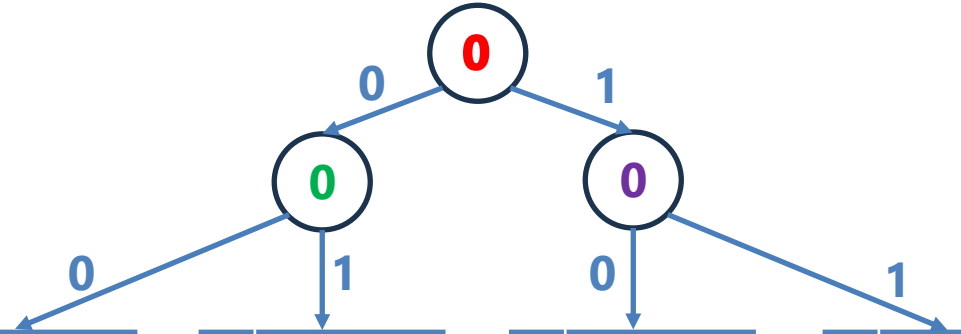


PLRU (Pseudo-LRU) Replacement

- Our read history is: 000000, 000001, 00010, 000011.

PLRU	V	Tag	V	Tag	V	Tag	V	Tag
000	0		0		0		0	

- Our next read is to 000011. Which block should we choose?
  - Then update PLRU bits so they point opposite direction traversed.



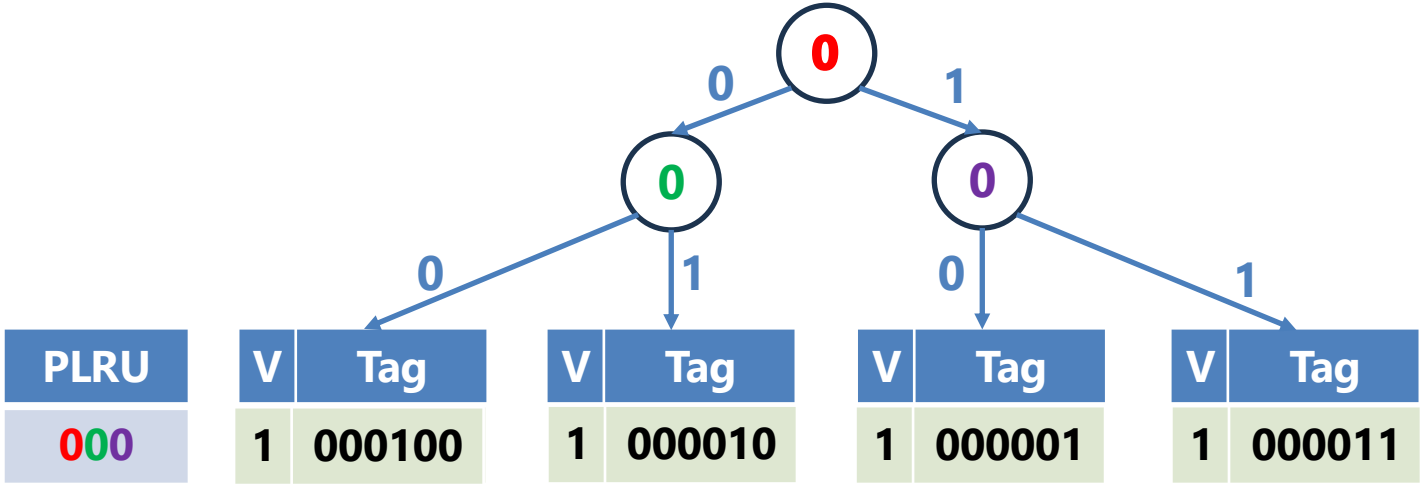
PLRU	V	Tag	V	Tag	V	Tag	V	Tag
000	1	000000	1	000010	1	000001	1	000011

PLRU (Pseudo-LRU) Replacement

- Our read history is: 000000, 000001, 00010, 000011.

PLRU	V	Tag	V	Tag	V	Tag	V	Tag
000	0		0		0		0	

- Say our next read is to 000100.
  - Then we correctly choose the LRU block!

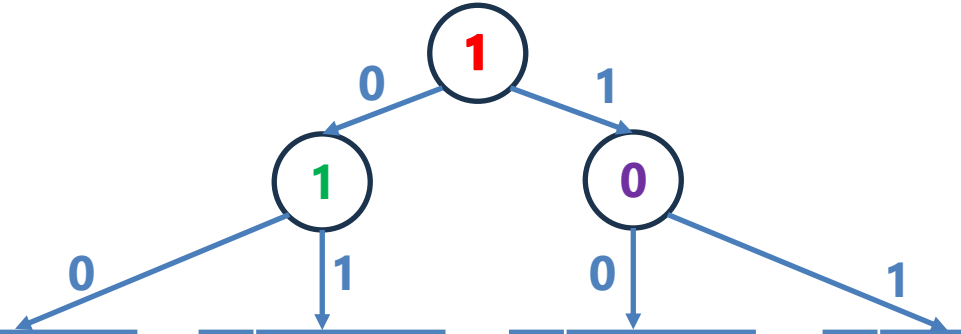


PLRU (Pseudo-LRU) Replacement

- Our read history is: 000000, 000001, 00010, 000011.

PLRU	V	Tag	V	Tag	V	Tag	V	Tag
000	0		0		0		0	

- Say our next read is to 000100.
  - Then we update PLRU bits to point to next LRU block.



PLRU	V	Tag	V	Tag	V	Tag	V	Tag
110	1	000100	1	000010	1	000001	1	000011